

第八届中国研究生人工智能创新大赛

技术文档

TARDIS

Transport-Aligned Residual Diffusion in Innovation Subspaces

团队名称	我是真没招了
参赛组别	企业赛题——基于扩散模型的实时视频生成算法时域一致性优化
文档范围	部署、训练、推理协议、SSH 反向代理与运维规范
文档版本	V1.2

2026 年 8 月

目录

1	技术范围与设计原则	1
2	系统总览	1
2.1	三个工程边界	2
3	TARDIS 方法	2
4	项目结构	4
5	数据集与 benchmark	4
6	GPU 服务端部署	5
6.1	环境	5
6.2	数据盘和缓存	6
6.3	训练	6
6.4	全量推理评测	6
6.5	Prompt-only 生成	7
7	Web-server 与 SSH 反向代理服务	7
8	TARDIS Studio 桌面客户端	8
9	端到端部署口径与 TARDIS 推理服务协议	9
9.1	设备、软件与正式参数	10
9.2	预训练衔接、后训练与微调细节	11
9.2.1	预训练边界与参数所有权	11
9.2.2	预训练衔接与时序适配命令	11
9.2.3	六阶段后训练课程	12
9.2.4	微调、恢复与可选奖励校准	13
9.2.5	优化器、AMP 与 EMA	14
9.2.6	正式推理参数与命令	15
9.3	服务拓扑与认证边界	16
9.4	上行协议：客户端或 Web Server → TARDIS 推理服务端	17

9.5	下行协议：状态、结果与错误	19
9.6	本地客户端适配协议	20
9.7	Nginx 与 SSH 反向代理服务	21
9.8	数据集、数值 benchmark 与视觉参考模型	22
10	输出、日志与可复现性	24
11	研究实验脚本边界	24
12	故障排查	24
12.1	找不到数据或 manifest	24
12.2	找不到 checkpoint	25
12.3	CUDA out of memory	25
12.4	Web-server 端口冲突	25
12.5	客户端无法轮询或下载	25
13	安全与许可证	25
14	引用与相关文档	25

1 技术范围与设计原则

[GitHub Repository](#) • [main branch](#)

Transport-Aligned Residual Diffusion in Innovation Subspaces

TARDIS 是一个面向连续视频生成的完整工程：GPU 服务端负责训练、验证、评测和 **prompt-only** 推理；**web-server** 提供轻量 HTTP 服务和 **nginx** 前置，可作为 SSH 反向代理服务的运维前置；**tardis-client** 提供可在 Windows 上运行的 Electron 桌面创作客户端。

项目的核心原则是：

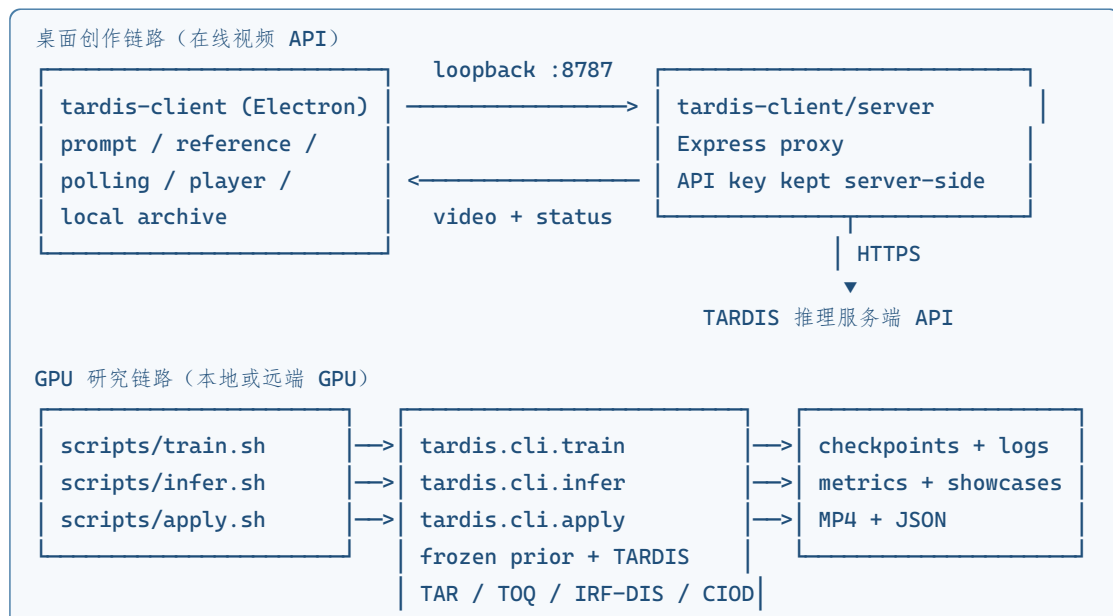
先传输可预测世界，再只扩散不可预测事件。

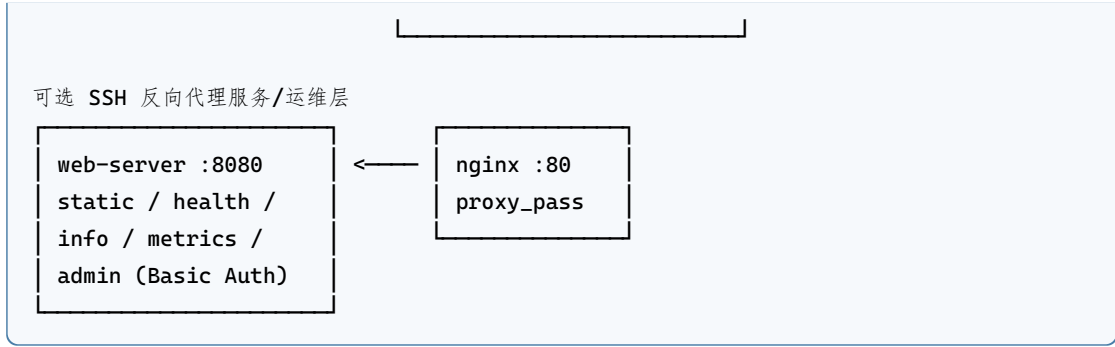
相邻视频帧中的背景、主体和纹理通常可以由历史状态和运动传输解释。TARDIS 先把上一帧生成状态对齐到当前坐标系，再在传输轨道的法向创新子空间中进行稀疏残差扩散，将预算集中到真正需要更新的区域。

本技术文档覆盖模型训练、验证与推理运行口径，异步推理服务的上下行协议，桌面客户端适配规则，以及 Nginx 与 SSH 反向代理服务的部署、安全和故障恢复边界。所有命令和参数均按训练节点、推理节点、公网前置和本地客户端四类角色组织。

2 系统总览

仓库包含两条彼此独立的运行链路。桌面创作链路面向在线视频生成，使用客户端内置的 Express 代理访问 TARDIS 推理服务端；GPU 研究链路面向 TARDIS 的训练、评测和 **prompt-only** 推理，直接运行根目录下的 Python CLI。**web-server** 是可选的 HTTP/运维前置，不是当前客户端的生成 API，也不包含 GPU 推理代码。





2.1 三个工程边界

组件	目录	作用	默认入口
TARDIS GPU 引擎	根目录 tardis/ 、 scripts/	训练、验证、全量评测、prompt-only 生成	bashscripts/train.sh 、 infer.sh 、 apply.sh
SSH 反向代理服务与前置服务	web-server/	JDK 17 HTTP 服务、静态资源、健康检查、指标、限流、Basic Auth、nginx 反代；不提供 /api/generations	scripts/start.ps1 或 Docker Compose
桌面客户端	tardis-client/	prompt、参考图、进度轮询、播放、创作记录和本地归档	npmrundesktop:dev 或打包 EXE

服务端本体是 GPU 上的 Python CLI/推理引擎，不是 FastAPI/Flask 常驻 HTTP API。客户端的生成请求由 [tardis-client/server](#) 直接转发到 TARDIS 推理服务端的异步接口；[web-server](#) 只承担静态资源和运维接口。若云端部署需要长期 HTTP 入口或反向 SSH 通道，应在部署环境中额外配置受限隧道，并明确映射到实际 API 服务。

3 TARDIS 方法

TARDIS 将时序生成拆成两个互补部分：

1. **TAR (Transport-Aligned Residualization)**：从文本和因果状态预测运动，传输上一帧 latent、短期状态和 anchor，并构造当前帧相对于 transport prior 的残差。

2. **DIS (Diffusion in Innovation Subspaces)**: 通过可见性校准的创新风险路由器选择 `active patch`, 将残差投影到 `transport orbit` 的法向子空间, 仅对风险区域的创新 `token` 运行稀疏 `residual DiT`。

推理闭环:

```
prompt
-> frozen text encoder / TARDIS first-frame prior
-> frame 0
-> prompt-conditioned motion scaffold
-> motion/state transport
-> transport-orbit quotient projector
-> innovation risk field + proper time
-> lite tangent corrector + sparse normal residual diffusion
-> causal state update
-> next frame
```

主要模块位于 `tardis/models/`:

模块	责任
<code>priors.py</code>	冻结 VAE、文本编码器和首帧图像先验
<code>motion.py</code>	<code>source-motion teacher</code> 与 <code>prompt-conditioned motion scaffold</code>
<code>transport.py</code>	<code>latent warp</code> 、 <code>visibility</code> 加权、历史状态传输
<code>quotient.py</code>	<code>transport Jacobian</code> 轨道、 <code>tangent/normal</code> 残差分解
<code>router.py</code> 、 <code>clock.py</code>	创新风险、 <code>active patch</code> 和事件时间预算
<code>residual.py</code>	<code>lite tangent corrector</code> 与 <code>sparse residual DiT</code>
<code>state.py</code>	<code>causal state</code> 、 <code>anchor</code> 、 <code>scene-cut reset</code> 和长期记忆
<code>tardis.py</code> 、 <code>factory.py</code>	主模型编排与 <code>checkpoint-compatible</code> 装配

训练目标同时包含官方 `TC`、`LPIPS`、残差扩散、`transport`、`flow/visibility`、`router`、`drift`、文本和因果创新算子蒸馏项。完整默认值见 `scripts/train.sh` 和 `docs/train.md`。

4 项目结构

```

project/
├── tardis/           # TARDIS Python package
│   ├── cli/         # train / infer / apply / runtime
│   ├── data/        # dataset contracts, manifest, archive, split
│   ├── models/       # TAR, TOQ, IRF/DIS, state updater
│   ├── training/     # objective, losses, curriculum, EMA, distill
│   ├── metrics/      # TC, LPIPS, FVD, FID, CLIPScore, SSIM
│   ├── experiments/  # benchmarks, ablations, diagnostics, reports
│   └── utils/        # checkpoint, video I/O, resources, randomness
├── scripts/         # deployment-oriented shell entry points
├── docs/            # train/infer/apply/dataset documentation
│   └── demo/        # README GIFs and selected visual evidence
├── tests/           # unit and integration tests
├── data/            # .gitkeep only; large datasets stay on data disk
├── checkpoints/     # .gitkeep only; large weights stay on data disk
├── outputs/         # .gitkeep only; generated outputs stay on data disk
├── web-server/      # JDK HTTP service and nginx configuration
├── tardis-client/   # Electron + React desktop application
├── datasets.txt     # canonical dataset roots (edit per machine)
├── pyproject.toml   # Python package and dependency contract
├── CITATION.cff     # citation metadata
└── LICENSE          # Apache License 2.0 for project code

```

5 数据集与 benchmark

项目内部固定使用三个 canonical dataset 名称：

canonical name	上游数据源与引用	默认 manifest root
dataverse	Vchitect T2V DataVerse	Vchitect_T2V_DataVerse
openvid	OpenVid-1M	OpenVid-1M
seedance	seedance-2-prompts-datasets	seedance-2-prompts-datasets

默认 datasets.txt 中记录的是：

```

/home/TARDIS/data/Vchitect_T2V_DataVerse
/home/TARDIS/data/OpenVid-1M
/home/TARDIS/data/seedance-2-prompts-datasets

```

部署到其他机器时，推荐复制路径文件并设置 `TARDIS_DATASETS_FILE`，不要把大规模视频复制进 Git。标准 split 使用 train/validation/test 三段，默认 split seed 为 3407；记录数和 manifest 版本应以目标数据盘上的 `tardis_manifest.jsonl` 与 `curation_report.json` 为准。

正式 benchmark 设置是 3 个数据集、TARDIS 与 9 个外部对比基线：

#	方法标识	SOTA 对比模型与引用
1	animatediff_lightning	AnimateDiff-Lightning
2	sd_turbo_independent	SD-Turbo 独立逐帧基线
3	text2video_zero	Text2Video-Zero
4	streamdiffusion_img2img	StreamDiffusion image-to-video path
5	rerender_flow	ReRender-A-Video
6	tokenflow_core	TokenFlow
7	vid2vid_zero_core	vid2vid-zero
8	controlvideo_canny	ControlVideo Canny condition
9	stablevideo_propagation	StableVideo propagation

后六类通常属于 source-conditioned 对比，不能与 prompt-only [apply](#) 结果混写。实现和分组位于 [tardis/experiments/benchmark.py](#)。

6 GPU 服务端部署

6.1 环境

- Linux + NVIDIA CUDA GPU；训练和推理默认使用 `bf16`。
- Python `>=3.12`。
- PyTorch `>=2.8,<2.9`，以及 `diffusers`、`transformers`、`accelerate`、`torchvision`、`av`、`opencv-python-headless`、`lpips`、`open-clip-torch`、`torchmetrics` 等。
- 可访问模型缓存和数据盘；默认基础先验由 TARDIS 推理服务端提供。

安装：

```
cd /path/to/project
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -e .
```

研究/代码质量依赖：

```
python -m pip install -e '[dev]'
```


6.2 数据盘和缓存

```
export TARDIS_STORAGE_ROOT=/root/autodl-tmp/TARDIS
export TARDIS_DATASETS_FILE=/path/to/project/datasets.txt
export TARDIS_CHECKPOINT_ROOT="$TARDIS_STORAGE_ROOT/checkpoints"
export TARDIS_OUTPUT_ROOT="$TARDIS_STORAGE_ROOT/outputs"
export HF_HOME="$TARDIS_STORAGE_ROOT/cache/huggingface"
export TORCH_HOME="$TARDIS_STORAGE_ROOT/cache/torch"
```

默认脚本会设置 `HF_ENDPOINT=https://hf-mirror.com` 和 `HF_HUB_DISABLE_XET=1`；正式实验应记录 manifest、checkpoint SHA-256、模型签名、分辨率、帧数、采样步数、精度和 GPU 型号。

6.3 训练

标准入口通过脚本位置推导根目录，提升服务端内容后不需要额外改路径：

```
cd /path/to/project
source .venv/bin/activate
TARDIS_DATASET=dataverse bash scripts/train.sh
```

切换数据集：

```
TARDIS_DATASET=openvid bash scripts/train.sh
TARDIS_DATASET=seedance bash scripts/train.sh
```

常用覆盖项：

```
TARDIS_EPOCHS=20 \
TARDIS_STEPS_PER_EPOCH=64 \
TARDIS_MICRO_BATCH_SIZE=2 \
TARDIS_GRADIENT_ACCUMULATION_STEPS=2 \
TARDIS_NUM_FRAMES=16 \
TARDIS_DIFFUSION_STEPS=2 \
TARDIS_ACTIVE_RATIO=0.35 \
bash scripts/train.sh
```

训练在 validation 上使用 TC/LPIPS 选择 best.pt，并保存 latest.pt、日志和统计；test 指标不参与 checkpoint 选择。

6.4 全量推理评测

```
TARDIS_DATASET=dataverse \
TARDIS_CHECKPOINT=/root/autodl-tmp/TARDIS/checkpoints/dataverse/<run>/best.pt \
bash scripts/infer.sh
```

infer 针对一个数据集完整 test split 生成指标和固定数量 showcase：

```
outputs/infer/<dataset>/<timestamp>/
└─ metrics.xlsx / metrics.csv
```

```
├─ per_video_details.csv / per_video_details.jsonl
├─ failures.jsonl
├─ latency.json / resources.json
└─ showcases/*.mp4
```

主指标为官方 TC 和 LPIPS; FVD、FID、CLIPScore、SSIM、warp error、长序列 drift 与延时统计用于诊断。只有在同一协议、同一数据划分和同一硬件上比较时,才应把数字写成性能结论。

6.5 Prompt-only 生成

apply 不读取 source video, 仅由 prompt 和首帧 prior 启动因果 rollout:

```
TARDIS_DATASET=dataverse \
TARDIS_CHECKPOINT=/root/autodl-tmp/TARDIS/checkpoints/dataverse/<run>/best.pt \
TARDIS_PROMPT='A small robot walks through a misty bamboo forest at sunrise' \
TARDIS_STYLE=cinematic \
TARDIS_DURATION=2 \
bash scripts/apply.sh
```

输出:

```
outputs/apply/<dataset>/<timestamp>/video.mp4
outputs/apply/<dataset>/<timestamp>/video.json
```

不要把 apply 的 prompt-only 输出写成 source-conditioned video editing。

7 Web-server 与 SSH 反向代理服务

web-server/ 是 JDK 17 HTTP 服务, 提供静态文件、健康检查、服务器信息、运行时指标、CORS、按 IP 限流和 Basic Auth 管理接口。nginx 配置将公网 80 端口反代到应用 8080 端口。

要求 JDK 17+、Maven 3.8+:

```
cd web-server
.\scripts\start.ps1
```

脚本默认执行 mvn package 后启动 target/tardis-webserver.jar; 需要跳过测试构建时:

```
.\scripts\start.ps1 -SkipTests
```

Linux/macOS 等价命令:

```
cd web-server
mvn package
java -jar target/tardis-webserver.jar
```

默认监听 0.0.0.0:8080。Docker + nginx:

```
cd web-server
docker compose up --build
```

HTTP 接口：

接口	作用
GET/	静态首页
GET/api/health	健康状态
GET/api/info	服务信息
GET/api/metrics	请求计数、活跃请求和按路径统计
GET/admin/info	Basic Auth 管理信息
GET/admin/metrics	受保护的运行时指标

仓库中的 JDK 服务本身不包含 GPU 推理逻辑。若部署环境将它作为 SSH 反向代理服务的 HTTP 前置，运维层可以建立受限隧道：

```
ssh -N -T \
-o ExitOnForwardFailure=yes \
-R <gateway-port>:127.0.0.1:8080 \
<user>@<gateway-host>
```

端口、用户、密钥和绑定地址必须按安全组和 SSH 策略配置；不要把管理接口或 SSH 私钥暴露到公网。`nginx.conf` 中的 `proxy_pass` 目标也应按实际隧道端口修改。

8 TARDIS Studio 桌面客户端

客户端是 Electron + React + Vite 应用，支持：

- prompt 必填，最多 512 个字符；
- PNG/JPG/JPEG 参考图上传，并在对话框上方即时预览；
- 画幅、时长、FPS、质量和音效选项；
- 异步任务提交与 `/api/generations/:id` 轮询进度；
- 完成后在桌面窗口内播放视频；
- 侧边栏创作记录；
- 下载视频、封面和 `manifest.json` 到本地归档；
- Electron `safeStorage` 保存 API Key，渲染层不接触明文密钥。

开发运行：

```
cd tardis-client
npm install
npm run desktop:dev
```

只调试网页界面时可使用 `npm:dev`；完整桌面链路使用 `desktop:dev`。

服务端 API 密钥只应放在本机环境变量或客户端设置面板中：

```
$env:TARDIS_INFERENCE_API_KEY = '<your-service-key>'
npm run desktop:dev
```

客户端本地 Express 代理默认监听 127.0.0.1:8787，代理将异步请求发送到 TARDIS 推理服务端的视频接口。渲染层只访问本地代理，不直接持有服务端密钥。

构建桌面包：

```
npm run build
npm run desktop:dist
```

Windows 产物位于 `tardis-client/release/win-unpacked/`，可直接启动 `TARDISStudio.exe`。本地归档默认位于 `%APPDATA%\tardis-video-studio\archives`，可用 `TARDIS_ARCHIVE_DIR` 覆盖。

客户端 API 契约：

```
POST http://127.0.0.1:8787/api/generations
Content-Type: application/json
```

```
{
  "prompt": "A cinematic robot walks through a rainy neon street",
  "imageData": "data:image/png;base64,...",
  "settings": {
    "quality": "speed",
    "size": "1280x720",
    "fps": 30,
    "duration": 5,
    "withAudio": false
  }
}
```

创建任务后轮询：

```
GET http://127.0.0.1:8787/api/generations/<task-id>
```

终态为 `PROCESSING`、`SUCCESS` 或 `FAIL`。云端 `videoUrl/coverUrl` 是临时地址，客户端成功后会立即下载到本地归档。完整字段约束见 [tardis-client/server/API_CONTRACT.md](#)。

9 端到端部署口径与 TARDIS 推理服务协议

本节是技术文档的规范版本。它把“研究 CLI”“推理服务端”“SSH 反向代理服务”和“桌面客户端”分成四个可独立替换的边界，并明确哪些参数是正式验收口径、哪些只是脚本的 smoke profile。服务端名称统一为 **TARDIS 推理服务端**；公网节点只做 TLS、鉴权、限流和转发，不执行模型计算。

9.1 设备、软件与正式参数

正式 **profile** 按角色固定如下，长参数采用列表记录，避免把一串不可断开的配置塞进表格：

- **训练节点**：NVIDIA RTX 4090（24 GB）；Python 3.12.3、PyTorch 2.8、bf16 AMP、micro batch 1、梯度累计 4、validation batch 2、EMA 0.999、gradient checkpointing。
- **推理节点**：NVIDIA RTX 5060（8 GB）；batch 1、两次 endpoint update、active ratio 0.35、VAE slicing、因果状态缓存，显存不足时 fp16 fallback。
- **公网前置**：Linux CPU 节点；Nginx 1.24+、TLS 443、keepalive 16、请求体 10 MB、read timeout 120 s、限流 20 r/s（burst 40）。
- **客户端**：Windows 10/11；Electron + React + Vite，本地代理 127.0.0.1:8787，播放器、创作记录和本地 MP4/封面/manifest 归档。

脚本中的可选 **smoke profile**（micro batch 2、梯度累计 2、validation batch 8）只用于快速检查路径和张量形状，不用于正式性能或质量结论。正式训练命令如下：

```
cd /path/to/project
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -e .
export TARDIS_STORAGE_ROOT=/root/autodl-tmp/TARDIS
export TARDIS_DATASETS_FILE=/path/to/project/datasets.txt
export TARDIS_CHECKPOINT_ROOT="$TARDIS_STORAGE_ROOT/checkpoints"
export TARDIS_OUTPUT_ROOT="$TARDIS_STORAGE_ROOT/outputs"
TARDIS_DATASET=dataverse \
TARDIS_MICRO_BATCH_SIZE=1 \
TARDIS_GRADIENT_ACCUMULATION_STEPS=4 \
TARDIS_VALIDATION_BATCH_SIZE=2 \
TARDIS_EPOCHS=20 \
bash scripts/train.sh
```

验证、全量评测和 **prompt-only** 推理：

```
TARDIS_DATASET=dataverse \
TARDIS_CHECKPOINT="$TARDIS_CHECKPOINT_ROOT/dataverse/<run>/best.pt" \
bash scripts/infer.sh

TARDIS_DATASET=dataverse \
TARDIS_CHECKPOINT="$TARDIS_CHECKPOINT_ROOT/dataverse/<run>/best.pt" \
TARDIS_PROMPT='A cinematic robot walks through a rainy neon street' \
TARDIS_STYLE=cinematic TARDIS_DURATION=2 \
bash scripts/apply.sh
```

infer.sh 只读取锁定 **test split**，输出 **metrics.csv**、逐视频明细、失败记录、延迟和显存 JSON；**apply.sh** 不读取 **source video**，输出 **video.mp4** 与 **video.json**。正式选择规则为 validation TC/LPIPS 加权分数，**test** 标签不参与训练、调参或 **checkpoint** 选择。

9.2 预训练衔接、后训练与微调细节

9.2.1 预训练边界与参数所有权

TARDIS 的像素生成器并不在本项目中重新预训练。外部预训练 VAE、文本编码器和首帧先验作为冻结模块加载，并以 `bf16` 驻留在 GPU；本项目真正训练的是时域增量参数：`motion`、`transport`、`state`、`router`、`lite_corrector`、`keyframe_residual_dit`、`residual_teacher/student` 和 `metric_adapter`。这样可以把“像素先验的语义能力”和“时域一致性模块的增量能力”分开审计，也避免三个数据集之间发生权重泄漏。每个数据集单独建立运行目录、优化器状态和 EMA，不交叉合并 checkpoint。

正式数据契约是 `512x512` 空间分辨率、连续 16 帧训练窗口、30 FPS metadata、固定划分子 `3407`。训练 loader 在窗口内随机水平翻转，validation/test 禁止随机增强；测试标签只在最终 `infer` 阶段读取。模型结构签名必须包含分辨率、帧数、latent channels、patch size、hidden size、层数、头数、active ratio、商空间阈值和扩散步数，任何一项不一致都拒绝恢复或加载。

阶段	可训练模块	冻结模块	输入窗口	目的
预训练衔接	<code>keyframe_residual_dit</code> 、 <code>lite_corrector</code>	VAE、文本编码器、首帧先验及其余时域模块	1 帧	让增量分支从 identity-preserving 初值起步
时序适配	<code>motion</code> 、 <code>transport</code> 、 <code>state</code> 、 <code>lite</code>	像素先验	16 帧	学习运动传输、可见性和状态更新
后训练	<code>router</code> 、 <code>residual_teacher/student</code> 、 <code>metric_adapter</code> 逐阶段解冻	像素先验	16 帧	闭环、蒸馏和 TC/LPIPS 对齐
部署推理	无梯度、EMA shadow	全部参数固定	16 帧或按时长展开	以两步 endpoint 轨迹生成 MP4

9.2.2 预训练衔接与时序适配命令

以下命令是可复现的两步入口。第一步只训练关键帧增量分支，第二步从第一步的 EMA 权重进行完整时序适配；`TARDIS_WARM_START_USE_EMA=1` 明确指定加载 shadow 权重，而不是加载训练中间态。

```
# P0: keyframe adapter warm-up (one dataset, one RTX 4090)
cd /path/to/project
source .venv/bin/activate
export TARDIS_STORAGE_ROOT=/root/autodl-tmp/TARDIS
```

```

export TARDIS_DATASETS_FILE=/path/to/project/datasets.txt
export TARDIS_CHECKPOINT_ROOT="$TARDIS_STORAGE_ROOT/checkpoints"
export TARDIS_OUTPUT_ROOT="$TARDIS_STORAGE_ROOT/outputs"
TARDIS_DATASET=dataverse \
TARDIS_TRAIN_MODE=keyframe_only \
TARDIS_CURRICULUM_PROFILE=transport \
TARDIS_EPOCHS=4 \
TARDIS_STEPS_PER_EPOCH=64 \
TARDIS_MICRO_BATCH_SIZE=1 \
TARDIS_GRADIENT_ACCUMULATION_STEPS=4 \
TARDIS_VALIDATION_BATCH_SIZE=2 \
TARDIS_PRECISION=bf16 \
bash scripts/train.sh

```

```

# P1: full temporal adaptation from the P0 EMA checkpoint
TARDIS_DATASET=dataverse \
TARDIS_WARM_START="$TARDIS_CHECKPOINT_ROOT/dataverse/<p0-run>/best.pt" \
TARDIS_WARM_START_USE_EMA=1 \
TARDIS_TRAIN_MODE=full_temporal \
TARDIS_CURRICULUM_PROFILE=full \
TARDIS_EPOCHS=20 \
TARDIS_STEPS_PER_EPOCH=64 \
TARDIS_MICRO_BATCH_SIZE=1 \
TARDIS_GRADIENT_ACCUMULATION_STEPS=4 \
TARDIS_VALIDATION_BATCH_SIZE=2 \
TARDIS_NUM_FRAMES=16 \
TARDIS_DIFFUSION_STEPS=2 \
TARDIS_ACTIVE_RATIO=0.35 \
TARDIS_EMA_DECAY=0.999 \
bash scripts/train.sh

```

`TARDIS_TRAIN_MODE=keyframe_only` 只把 keyframe residual DiT 和 lite corrector 设为可训练，并把训练 loader 的窗口缩为 1 帧；`full_temporal` 才会按照课程表逐步解冻完整时域模块。两步运行都必须记录 `run_manifest.json`、数据 manifest hash、预训练权重 hash、结构签名和有效参数，以便后续审计。OpenVid 与 Seedance 只需将 `TARDIS_DATASET` 替换为对应名称，不能在同一次进程里混合来源。

9.2.3 六阶段后训练课程

后训练以 optimizer step 而不是 micro-batch 计数。总预算为 $20 \times 64 / 4 = 320$ 个 optimizer steps，按 5%/5%/10%/20%/20%/40% 分配，每一阶段至少保留一个 step。teacher-forcing 只控制历史状态来源，不改变验证和测试数据；从闭环阶段开始逐渐用模型自己的上一帧替代真实帧。

阶段	预算	teacher forcing	residual steps	新增目标与可训练组
transport_warmup	5%	1.00	0	diffusion、transport、flow、visibility、lite; motion / transport / state
router_calibration	5%	1.00	0	加入 router、survival、budget; 校准 risk field 和 active-token
residual_teacher	10%	1.00	4	加入 residual teacher; 学习法向创新残差
closed_loop	20%	1.00 -> 0.25	4	加入 warp 和 drift; 使用自身历史状态
crcd	20%	0.25 -> 0	1	teacher/student 因果残差蒸馏, stop-gradient target
metric_alignment	40%	0	1	加入 TC、LPIPS、text 和 metric adapter; 选择 best.pt

损失权重在各数据集之间保持不变: TC=5.0、LPIPS=3.0、diffusion=1.0、keyframe=1.0、residual=1.0、transport=1.0、flow=0.1、visibility=0.1、router=0.2、survival=0.2、lite=0.2、budget=0.05、warp=0.2、drift=0.1、crcd=1.0、text=0.1。metric_alignment 阶段的 checkpoint 选择只使用 validation 上的 TC/LPIPS 组合分数; FVD、FID、CLIPScore 和 SSIM 仅写入诊断日志。

9.2.4 微调、恢复与可选奖励校准

微调是“权重衔接后在新来源或新域上继续优化”，不是重新加载全部优化器状态。跨数据集微调必须显式设置 `TARDIS_ALLOW_CROSS_DATASET_WARM_START=1`，否则 loader 会拒绝；正式结果推荐使用同数据集 warm start，并从 `metric_alignment` profile 开始：


```
# Dataset-specific fine-tuning on validation data only
TARDIS_DATASET=openvid \
TARDIS_WARM_START="$TARDIS_CHECKPOINT_ROOT/dataverse/<run>/best.pt" \
TARDIS_ALLOW_CROSS_DATASET_WARM_START=1 \
TARDIS_WARM_START_USE_EMA=1 \
TARDIS_CURRICULUM_PROFILE=metric_alignment \
TARDIS_EPOCHS=6 \
TARDIS_STEPS_PER_EPOCH=64 \
TARDIS_MICRO_BATCH_SIZE=1 \
TARDIS_GRADIENT_ACCUMULATION_STEPS=4 \
TARDIS_VALIDATION_BATCH_SIZE=2 \
bash scripts/train.sh
```

精确续训使用 `TARDIS_RESUME`，要求数据集、world size、结构签名和 precision 与原运行一致；它会恢复 model、optimizer、scheduler、AMP scaler、EMA、课程游标、采样器状态和三套随机数状态。周期 checkpoint 每 256 个 micro-batch 在累计边界原子写入，epoch 末尾再写一次；只有完整 validation 改善时才替换 `best.pt`。

```
# Exact resume: optimizer and curriculum state are restored
TARDIS_DATASET=openvid \
TARDIS_RESUME="$TARDIS_CHECKPOINT_ROOT/openvid/<run>/latest.pt" \
TARDIS_PRECISION=bf16 \
bash scripts/train.sh
```

奖励校准是可选的离线后训练环节，不是默认部署路径。它只更新 router 和 `residual_scale_adapter`，冻结像素先验、主干扩散权重和两步 endpoint 轨迹。状态包含运动特征、可见性、transport latent、历史缓存和文本条件；动作包含 active mask、残差尺度、proper-time budget 与 scene-cut refresh。奖励按 TC/LPIPS 改善、long-term drift、active-token/VAE cost 和 refresh error 组合，使用 group-relative advantage、policy clip 0.2 和 KL 约束 0.01。校准数据只能来自 train/validation rollout；若 validation 组合分数变差，立即回退到 EMA `best.pt`。交付仓库没有把该可选环节伪装成独立生产 CLI，因此任何启用它的运行都必须在 manifest 中记录 adapter diff、奖励版本、KL 约束和回退点。

9.2.5 优化器、AMP 与 EMA

正式训练在 RTX 4090 (24 GB) 上使用 AdamW，峰值学习率 $1e-4$ 、weight decay $1e-2$ ，前 64 个 optimizer steps 线性 warmup，之后 cosine decay，global-norm clipping 1.0。正式 profile 为 micro batch 1、gradient accumulation 4、validation batch 2、8 个 worker、prefetch factor 4；单卡有效 batch 为 $1 \times 1 \times 4 = 4$ 。bf16 autocast 覆盖前向、损失和反向，FP32 保留在 master weights、归一化统计、Gram 阻尼、梯度范数和指标聚合；只有明确切换到 fp16 时才启用 GradScaler。每个有效 optimizer step 更新时域参数 EMA，decay=0.999，冻结先验不进入 shadow。验证时临时交换 EMA 参数，完成后恢复即时参数，避免污染继续训练状态。

非有限损失、梯度溢出、解码失败和 OOM 均写入 non-finite ledger；不得静默跳过样本。latest.pt 采用临时文件加原子替换，包含模型和所有恢复状态；best.pt 额

外记录 validation 样本数、TC、LPIPS、组合分数、选择时间、checkpoint SHA-256 和硬件信息。

9.2.6 正式推理参数与命令

推理节点固定为 NVIDIA RTX 5060 (8 GB), batch=1, 默认使用 EMA、bf16、VAE slicing、因果状态缓存、两次 endpoint update 和 active ratio=0.35。推理结构参数必须与训练签名一致；可改变 prompt、style、seed 和 duration, 但不应在正式报告中临时改变 hidden size、层数或 quotient 阈值。关键 runtime 参数如下：

参数	正式值	作用与约束
height, width	512, 512	与 checkpoint 结构签名一致
num_frames	16	infer 的评测窗口长度
fps	30	MP4 metadata; 不是吞吐承诺
latent_channels	4	VAE latent 通道数
patch_size	2	token 化步长
hidden_size / num_layers / num_heads	512 / 8 / 8	时域 DiT 结构
active_ratio	0.35	风险路由保留的 active patch 比例
motion_max_flow_pixels	8.0	运动先验的最大像素流
transport_max_correction_pixels	0.25	单步传输修正上限
router_threshold / halo_radius	0.1 / 1	风险阈值和邻域扩张
state_anchor_decay	0.95	长期 anchor 衰减
scene_cut_threshold	0.98	场景切换 reset 阈值
quotient_regularization / rank_threshold	1e-4 / 1e-5	商空间稳定性
proper_time_maximum_hazard	20.0	innovation clock 上限
diffusion_steps / time sampling	2 / endpoint	部署轨迹，必须成对使用

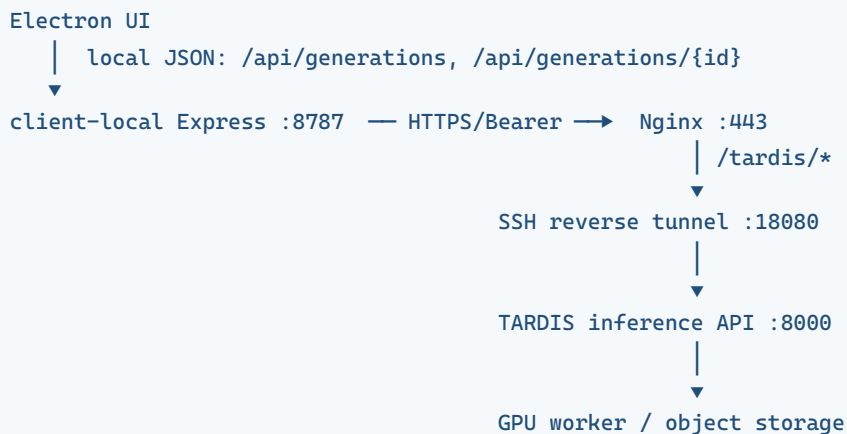
全量 test 评测和 prompt-only 生成分别调用 infer.sh 与 apply.sh。三个数据集必须分别执行，输出目录带数据集和时间戳，避免覆盖：

```
# I0: full locked test split on the RTX 5060
TARDIS_DATASET=dataverse \
TARDIS_CHECKPOINT="$TARDIS_CHECKPOINT_ROOT/dataverse/<run>/best.pt" \
TARDIS_USE_EMA=1 \
TARDIS_PRECISION=bf16 \
TARDIS_NUM_WORKERS=8 \
TARDIS_PREFETCH_FACTOR=4 \
TARDIS_SHOWCASE_COUNT=5 \
TARDIS_RESUME_METRICS=1 \
bash scripts/infer.sh
```

```
# I1: one prompt-only causal rollout
TARDIS_DATASET=dataverse \
TARDIS_CHECKPOINT="$TARDIS_CHECKPOINT_ROOT/dataverse/<run>/best.pt" \
TARDIS_PROMPT='A cinematic robot walks through a rainy neon street' \
TARDIS_STYLE='cinematic, stable camera, coherent motion' \
TARDIS_DURATION=2 \
TARDIS_HEIGHT=512 TARDIS_WIDTH=512 TARDIS_FPS=30 \
TARDIS_SEED=3407 TARDIS_USE_EMA=1 TARDIS_PRECISION=bf16 \
TARDIS_DIFFUSION_STEPS=2 TARDIS_ACTIVE_RATIO=0.35 \
bash scripts/apply.sh
```

`infer.sh` 对测试 `split` 的每条记录执行 `batch-1` 生成，默认保存 5 个可复现 `showcase`，并写入六项指标、逐视频明细、失败记录、延迟和资源 JSON。`apply.sh` 不读取 `source video`，帧数按 `round(durationxfps)` 计算，写入 `video.mp4` 和包含 `prompt`、`style`、`seed`、结构签名、权重 `SHA-256`、生成/编码耗时的 `video.json`。两条路径都在生成前后执行 `CUDA` 同步；推理速度报告区分首帧、`steady-state`、`full-refresh`、编码和传输时间，不能用 `metadata FPS` 替代实测吞吐。

9.3 服务拓扑与认证边界



客户端只保存本地任务记录，不接触 GPU 文件系统。Bearer 凭据从服务端环境变量或桌面端安全存储读取；绝不放在 URL、日志、截图或前端 `bundle`。每个请求都应携带 `X-Request-ID`，创建任务建议同时携带 `Idempotency-Key`，以避免网络重试产生重复推理。

9.4 上行协议：客户端或 Web Server → TARDIS 推理服务端

规范 Base URL 为 <https://<gateway-host>/tardis>。网关剥离 `/tardis` 前缀后，将请求转发到内部 `/v1`。公共请求头：

Header	必填	约束
<code>Authorization</code>	是	<code>Bearer<token></code> ，不允许查询参数携带令牌
<code>Content-Type</code>	JSON 必填	<code>application/json; charset=utf-8</code>
<code>Accept</code>	否	JSON 默认 <code>application/json</code> ，下载时为 <code>video/mp4</code>
<code>X-Request-ID</code>	否	由调用方提供或由网关生成并回传
<code>Idempotency-Key</code>	创建建议	同一 key 在保留窗口内只创建一个任务

创建任务：

```
POST /tardis/v1/videos
Authorization: Bearer <token>
Content-Type: application/json
Idempotency-Key: demo-20260827-0001
```

字段	类型	必填	默认/范围	说明
model	string	否	tardis	当前服务端模型标识
prompt	string	是	1–512 字符	正向文本描述，首尾空白会被去除
negative_prompt	string	否	服务默认	负向约束
num_inference_steps	integer	否	2（部署）	必须为正整数；能力接口给出上限
guidance_scale	number	否	7.5	文本引导强度
seed	integer	否	42	用于复现的随机种子
fps	integer	否	30	输出 metadata 帧率；真实吞吐单独测量
duration_seconds	number	否	5	不支持的值返回 422
output_format	string	否	mp4	当前仅支持 MP4
image_data	Data URL	否	-	PNG/JPEG，解码后不超过 5 MB
metadata	object	否	{}	追踪字段，不参与推理

示例：

```
{
  "model": "tardis",
  "prompt": "A white-robed heroine walks along a luminous mountain cliff, stable camera",
  "negative_prompt": "blurry, flicker, text, watermark",
  "num_inference_steps": 2,
  "guidance_scale": 7.5,
  "seed": 42,
  "fps": 30,
  "duration_seconds": 5,
  "output_format": "mp4"
}
```

成功返回 202Accepted，并带 Location:/tardis/v1/videos/{video_id}：

```
{
  "request_id": "req_01J8TARDIS0001",
  "data": {
    "id": "vid_01J8TARDIS0001",
    "object": "video",
    "status": "queued",
    "model": "tardis",
    "status_url": "/tardis/v1/videos/vid_01J8TARDIS0001",
    "requested_parameters": {"num_inference_steps": 2, "seed": 42},
    "effective_parameters": null
  }
}
```

9.5 下行协议：状态、结果与错误

状态查询：

```
GET /tardis/v1/videos/{video_id}
Authorization: Bearer <token>
Accept: application/json
```

任务状态只有 `queued`、`running`、`succeeded`、`failed`、`cancelled` 五种。运行中返回 `progress.percent`（0–99）和 `progress.stage`；成功返回有效参数、结果哈希和短期下载地址：

```
{
  "request_id": "req_01J8TARDIS0002",
  "data": {
    "id": "vid_01J8TARDIS0001",
    "status": "succeeded",
    "progress": {"percent": 100, "stage": "encoding"},
    "effective_parameters": {"width": 512, "height": 512, "fps": 30, "frames": 16, "seed": 42},
    "output": {"mime_type": "video/mp4", "download_url": "/tardis/v1/videos/vid_01J8TARDIS0001/content", "size_bytes": 12345678, "sha256": "<sha256>", "expires_at": "2026-08-28T12:00:00Z"},
    "error": null
  }
}
```

下载和取消：

```
GET /tardis/v1/videos/{video_id}/content
Authorization: Bearer <token>
Accept: video/mp4
```

下载响应必须包含 `Content-Type`、`Content-Length`、`Content-Disposition`、`Etag`；支持 `Range` 时允许 `206PartialContent`。`DELETE/tardis/v1/videos/{video_id}` 返回 202 并将未完成任务置为 `cancelled`；已完成任务返回 409。

统一错误包络：

```
{
  "request_id": "req_01J8TARDIS0003",
  "error": { "code": "validation_error", "message": "prompt is required", "details": [ { "field": "prompt", "reason": "required" } ], "retryable": false }
}
```

HTTP	code 示例	是否重试	说明
400	invalid_json	否	JSON 无法解析
401/403	unauthorized / forbidden	否	凭据或权限问题
404	not_found	否	任务不存在或结果已清理
409	idempotency_conflict / task_not_cancellable	按业务	状态冲突
413/422	payload_too_large / validation_error	否	请求体或参数不合法
429	rate_limited / queue_full	是	遵循 Retry-After
502/503/504	bad_gateway / upstream_unavailable / upstream_timeout	是	网关、隧道或 GPU 暂时不可用
500	internal_error	谨慎	服务端异常，带 request ID 联系运维

客户端建议每 2–5 秒轮询一次，单任务总等待上限 30 分钟；遇到 [429/503/504](#) 按指数退避（2、4、8、16、30 秒，上限 30 秒）并尊重 [Retry-After](#)。成功后立即下载，不要依赖会过期的远端 URL。

9.6 本地客户端适配协议

Electron 渲染层的本地代理仍使用短路径：

```
POST http://127.0.0.1:8787/api/generations
GET http://127.0.0.1:8787/api/generations/{task-id}
```

本地代理把 `prompt/imageData/settings` 映射到上述规范字段，把上游状态归一化为 `PROCESSING/SUCCESS/FAIL`。映射关系固定如下：

本地字段	规范字段	处理规则
<code>prompt</code>	<code>prompt</code>	必填，最长 512 字符
<code>imageData</code>	<code>image_data</code>	仅 PNG/JPEG Data URL， 5 MB 上限
<code>settings.size</code>	<code>width/height</code> 能力约束	不支持的尺寸由能力接口 拒绝
<code>settings.fps</code>	<code>fps</code>	30 或 60；以有效参数为 准
<code>settings.duration</code>	<code>duration_seconds</code>	5 或 10 秒；以有效参数 为准
<code>videoUrl</code>	<code>output.download_url</code>	成功后立即下载并计算 SHA-256

9.7 Nginx 与 SSH 反向代理服务

实际 `web-server` 应用默认绑定 `0.0.0.0:8080`，`backlog 64`、工作线程 8、限流 20 r/s、`burst 40`；当前应用接口为 `/api/health`、`/api/info`、`/api/metrics` 和受保护的 `/admin/*`。若将它作为公网运维前置，配置文件可采用以下最小安全模板，并把 `/tardis/` 指向 SSH 隧道回环端口：

```
worker_processes auto;
events { worker_connections 1024; }
http {
    include mime.types;
    sendfile on;
    keepalive_timeout 65;
    limit_req_zone $binary_remote_addr zone=tardis_rate:10m rate=20r/s;
    upstream tardis_admin { server 127.0.0.1:8080; keepalive 16; }
    server {
        listen 443 ssl http2;
        server_name <gateway-host>;
        ssl_certificate /etc/ssl/tardis/fullchain.pem;
        ssl_certificate_key /etc/ssl/tardis/privkey.pem;
        client_max_body_size 10m;
        location / { proxy_pass http://tardis_admin; proxy_http_version 1.1;
            proxy_set_header Host $host; proxy_set_header X-Request-ID $request_id;
            proxy_set_header X-Real-IP $remote_addr; proxy_set_header X-Forwarded-Proto https; }
        location /tardis/ { proxy_pass http://127.0.0.1:18080/; proxy_http_version 1.1;
            proxy_buffering off; proxy_read_timeout 120s; proxy_send_timeout 30s;
            proxy_set_header Authorization $http_authorization; proxy_set_header X-Request-ID $request_id;
        }
    }
}
```


配置检查与重载：

```
sudo nginx -t
sudo systemctl reload nginx
curl -fsS https://<gateway-host>/api/health
curl -fsS https://<gateway-host>/tardis/healthz
```

GPU 主机主动建立 SSH remote forwarding，网关端口只绑定回环地址：

```
ssh -NT \
-o ExitOnForwardFailure=yes \
-o ServerAliveInterval=30 \
-o ServerAliveCountMax=3 \
-o TCPKeepAlive=yes \
-R 127.0.0.1:18080:127.0.0.1:8000 \
tunnel@gateway.example.com
```

生产环境用 systemd 保持隧道：

```
[Unit]
Description=TARDIS inference reverse tunnel
After=network-online.target
Wants=network-online.target
[Service]
User=tardis
ExecStart=/usr/bin/ssh -NT -o ExitOnForwardFailure=yes -o ServerAliveInterval=30 -o
ServerAliveCountMax=3 -o StrictHostKeyChecking=yes -R 127.0.0.1:18080:127.0.0.1:8000
tunnel@gateway.example.com
Restart=always
RestartSec=5
[Install]
WantedBy=multi-user.target
```

将该单元保存为 `/etc/systemd/system/tardis-tunnel.service` 后执行：

```
sudo systemctl daemon-reload
sudo systemctl enable --now tardis-tunnel
sudo systemctl status tardis-tunnel
ss -lntp | grep 18080
```

网关 SSH 账号只允许 `remote` 转发（`AllowTcpForwardingremote`、`PermitListen127.0.0.1:18080`、`GatewayPortsno`），禁止密码登录和任意端口转发。隧道断开时 API 返回 `503upstream_unavailable`，恢复后不改变已经持久化的任务 ID。

9.8 数据集、数值 benchmark 与视觉参考模型

正式实验固定使用三个数据集：

数据集	上游链接	用途
DataVerse	Vchitect T2V DataVerse	主训练、验证和 test
OpenVid	OpenVid-1M	跨来源泛化与主对照
Seedance	seedance-2-prompts-datasets	风格与提示词覆盖

九个数值对比模型及其单一引用入口：

#	模型	链接	简介
1	StreamDiffusion	官方仓库	流式少步推理，速度基线
2	Rerender-A-Video	论文入口	关键帧传播与视频编辑
3	TokenFlow	官方仓库	扩散特征一致性
4	vid2vid-zero	官方仓库	零样本跨帧视频编辑
5	Text2Video-Zero	官方仓库	零样本文生视频
6	ControlVideo	官方仓库	结构条件控制
7	StableVideo	论文入口	帧间传播
8	AnimateDiff-Lightning	模型页面	少步运动先验
9	SD-Turbo independent	模型页面	逐帧下界基线

五个视觉参考模型用于同 prompt、同 seed、同帧位置的定性对照；它们不替代九模型数值 benchmark：

视觉模型	链接	定性观察重点
ZeroScope v2 576w	模型页面	低分辨率主体与背景稳定性
Pyramid Flow miniFLUX	项目页面	速度与构图平衡
Transformer-T2V 2B	论文与模型说明	2B 级文本视频生成基线
ModelScope T2V 1.7B	模型页面	早期开源 T2V 参考
Wan2.1 T2V 1.3B	项目页面	现代开源 T2V 参考

三种数据集、九个数值模型和五个视觉模型都使用固定 `split`、`seed`、分辨率、帧数和指标实现；`source-conditioned` 与 `prompt-only` 结果分开归档，禁止把两种协议混写。

10 输出、日志与可复现性

标准脚本将大型文件写入 `TARDIS_STORAGE_ROOT`:

```
$TARDIS_STORAGE_ROOT/
├─ cache/{huggingface,torch,xdg}/
├─ checkpoints/<dataset>/<run>/{latest.pt,best.pt}/
├─ outputs/
│   ├── train/<dataset>/<run>/
│   ├── infer/<dataset>/<run>/
│   └─ apply/<dataset>/<run>/{video.mp4,video.json}
```

复现实验至少保存：数据集 `revision`、manifest hash、split seed；checkpoint SHA-256 和模型结构签名；分辨率、帧数、FPS、采样步数、precision、GPU；TC/LPIPS 实现版本、每视频明细和失败样例；首帧、steady-state、full-refresh 的 latency、p50/p95、峰值显存和 `active ratio`。

当前仓库不会把“平均 FPS”自动解释为逐帧实时保证。是否满足 FPS 目标必须在目标硬件上包含 `motion`、`transport`、`quotient`、VAE 和 I/O 的端到端实测。

11 研究实验脚本边界

`scripts/train.sh`、`infer.sh`、`apply.sh` 是迁移后可直接使用的标准入口；它们通过 `SCRIPT_DIR` 推导根目录。`scripts/run_*` 和 `tardis/experiments/` 下部分 `benchmark`、队列和审计脚本仍保留 `/home/TARDIS` 或 `/root/autodl-tmp/TARDIS` 的研究环境假设，运行前应检查并覆盖绝对路径。

正式结果、抽帧图和代理指标来自独立证据包；README 中的 GIF 与缩略图是可视化演示，不应被当作全测试集统计。

12 故障排查

12.1 找不到数据或 manifest

```
cat "$TARDIS_DATASETS_FILE"
ls -lah /path/to/data-root/tardis_manifest.jsonl
```

确认 `TARDIS_DATASETS_FILE` 中的三个路径存在，并且每个源有 `manifest`、`curation report` 和媒体归档。

12.2 找不到 checkpoint

```
find "$TARDIS_CHECKPOINT_ROOT" -name best.pt -print
```

确保 `TARDIS_DATASET`、checkpoint 模型签名、分辨率、帧数、hidden size、层数和采样配置一致。

12.3 CUDA out of memory

降低 `TARDIS_MICRO_BATCH_SIZE`、`TARDIS_VALIDATION_BATCH_SIZE`，提高梯度累计，保持 `TARDIS_GRADIENT_CHECKPOINTING=1`，并确认 VAE decode chunk 没有超过显存容量。改变网络结构后不能直接复用不匹配 checkpoint。

12.4 Web-server 端口冲突

检查 `src/main/resources/application.properties` 的 `server.port`、nginx upstream 和 Docker 映射是否一致。不要同时启动 systemd、Docker 和手工 Java 进程，否则会争用 8080。

12.5 客户端无法轮询或下载

先检查本地 `127.0.0.1:8787` 代理，再确认服务端密钥有效、云端任务 ID 有效、临时视频 URL 尚未失效。客户端成功后应检查归档目录中的 `video.mp4`、`cover.jpg` 和 `manifest.json`。

13 安全与许可证

- 不要把 API Key、SSH 私钥、云端 token 或 `.env.local` 提交到仓库。
- 之前在聊天或日志中出现过的服务端密钥应立即轮换；README、截图和 GIF 不包含密钥。
- Web-server 默认 Basic Auth 凭据仅用于本地开发，生产环境必须改为强密码并限制管理接口来源。
- 项目代码采用 Apache License 2.0，见 [LICENSE](#)。数据集、基础模型、模型权重和第三方依赖遵循各自的许可证、访问政策和再分发限制。

14 引用与相关文档

若使用 TARDIS 代码或方法，请参考 [CITATION.cff](#)。视觉来源和对照链接见 [docs/demo/model_sources.txt](#)。

- [docs/train.md](#): 训练、验证和 checkpoint 选择
- [docs/infer.md](#): 全量 test split、指标和 showcase
- [docs/apply.md](#): prompt-only causal rollout

- [docs/datasets.md](#): manifest、归档和数据划分
- 本 README 已汇总桌面客户端、JDK HTTP 服务、nginx、Docker 和 SSH 反向代理服务说明
- [docs/demo/](#): 客户端演示结果、实机录屏 GIF、批量推理缩略图和视觉对比帧